CodeAlpha — Internet of Things Internship

Task 2: Sensor-Based Simulation

Temperature-Based IoT Alert System

Code Explanation & Simulation Documentation

1. Objective

The objective of this task is to design and simulate a simple, beginner-friendly Internet of Things (IoT) circuit that reads temperature data from a sensor and responds automatically using an LED. The project demonstrates the core IoT concept of "sense → decide → act" using an Arduino Uno, a TMP36 temperature sensor, and an LED, without requiring any physical hardware (simulated in Tinkercad Circuits).

A note on the sensor: Tinkercad Circuits does not include an LM35 part in its components library. The part it provides labeled "Temperature Sensor" is actually a TMP36 — the same 3-pin VCC / OUT / GND layout as an LM35, but a different output curve (it reads 500 mV at 0°C, then adds 10 mV per additional degree, instead of reading 0 mV at 0°C). This documentation, the circuit diagram, and the Arduino sketch all use the TMP36 and its correct conversion formula.

2. Components

Component	Quantity	Purpose
Arduino Uno	1	Microcontroller board that runs the program
TMP36 Temperature Sensor	1	Measures ambient temperature and outputs an analog voltage
LED	1	Visual alert indicator (ON when temperature is high)
Resistor (220 Ω)	1	Limits current through the LED to protect it
Jumper Wires	As needed	Connects components on the breadboard / Tinkercad canvas
Breadboard	1	Used to hold and connect components (virtual in Tinkercad)

3. Circuit Connections

The table below lists every wired connection in the circuit. The full labeled diagram is provided in circuit/circuit_diagram.png and below.

From	To	Wire Type
TMP36 – VCC pin	Arduino – 5V	Power (red)
TMP36 – GND pin	Arduino – GND	Ground (black)
TMP36 – OUT pin	Arduino – Analog pin A0	Signal (blue)
Arduino – Digital pin 8	Resistor (220 Ω) → LED Anode (+)	Signal (blue)
LED Cathode (–)	Arduino – GND	Ground (black)

Circuit Diagram

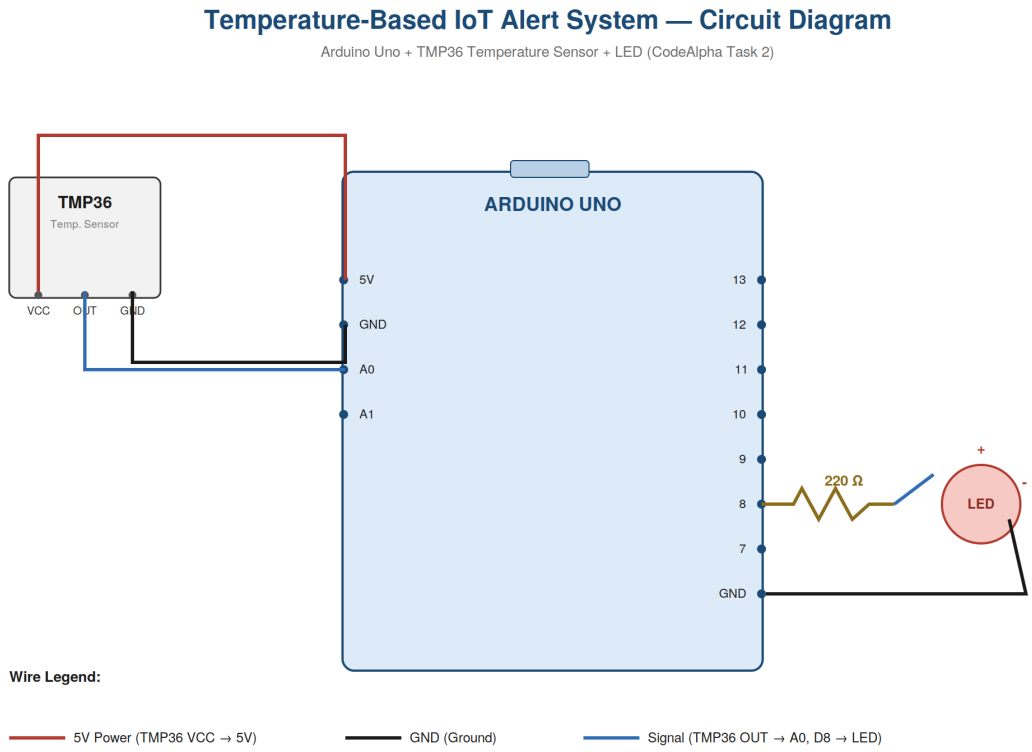

Figure 1: TMP36 wired to Arduino 5V / GND / A0; LED wired through a 220 Ω resistor to Digital Pin 8 and back to GND.

4. Working Principle

The TMP36 sensor outputs an analog voltage that increases linearly with temperature. It reads 500 millivolts at 0°C and adds 10 millivolts for every degree above that. The Arduino's analog-to-digital converter (ADC) reads this voltage on pin A0 as a raw value between 0 and 1023. The program converts this raw value into an actual voltage, subtracts the TMP36's 500 mV offset, and then converts the result into a temperature reading in degrees Celsius. This temperature is continuously printed to the Serial Monitor. The Arduino then compares the temperature against a fixed threshold of 30°C: if the temperature is 30°C or higher, Digital Pin 8 is set HIGH, lighting the LED as a visual alert; otherwise, the pin is set LOW and the LED stays off.

5. Algorithm

1. Start the program and initialize Serial communication at 9600 baud.
2. Set the LED pin (Digital Pin 8) as an OUTPUT and turn it OFF initially.
3. Read the raw analog value (0–1023) from the TMP36 sensor on pin A0.
4. Convert the raw value into a voltage using: voltage = (rawValue / 1024) × 5.0
5. Convert the voltage into temperature using: temperature (°C) = (voltage – 0.5) × 100
6. Print the calculated temperature to the Serial Monitor.
7. If temperature ≥ 30°C, set the LED pin HIGH (LED ON); otherwise set it LOW (LED OFF).
8. Print the current status (ALERT or Normal) to the Serial Monitor.
9. Wait for 1 second, then repeat from Step 3 (this repeats forever inside loop()).

6. Source Code Explanation

The complete sketch is provided in src/temperature_sensor.ino. A summary of each part of the code is given below.

Code Section	Explanation
Pin definitions (TMP36_PIN, LED_PIN)	Constants that store which Arduino pins the TMP36 and LED are connected to (A0 and Digital Pin 8).
setup()	Runs once at start-up. Initializes Serial communication at 9600 baud, sets the LED pin as OUTPUT, and makes sure the LED starts OFF.
analogRead(TMP36_PIN)	Reads the TMP36's analog output and returns a value from 0 to 1023, representing the sensor voltage.
voltage = (rawValue / 1024.0) × 5.0	Converts the raw ADC reading into an actual voltage, based on the Arduino's 5V reference and 10-bit ADC resolution.

temperatureC = (voltage – 0.5) × 100.0	Converts voltage into Celsius. The TMP36 outputs 500 mV at 0°C, so the 0.5V offset is subtracted before scaling by the 10 mV-per-degree slope.
Serial.print() / Serial.println()	Displays the temperature and system status (ALERT / Normal) on the Serial Monitor.
if (temperatureC ≥ TEMP_THRESHOLD)	Compares the temperature to the 30°C threshold and turns the LED ON or OFF using digitalWrite().
delay(1000)	Pauses for 1 second before the loop() function repeats, so readings update once per second.

7. Expected Output

When the simulation is running, the Serial Monitor should print a new temperature reading roughly once per second, similar to the example below:

```
Temperature-Based IoT Alert System Started...
=======================================
Temperature: 26.32 C
Status: Normal - LED is OFF
---------------------------------------
Temperature: 29.10 C
Status: Normal - LED is OFF
---------------------------------------
Temperature: 31.05 C
Status: ALERT - Temperature High! LED is ON
---------------------------------------
```

In Tinkercad, dragging the TMP36's temperature slider above 30°C should turn the LED on immediately, and dragging it back below 30°C should turn the LED off. Because of the TMP36's 500 mV offset, the sensor's simulated output voltage will read roughly 0.8 V at the 30°C threshold (not 0.3 V, as it would for an LM35). The exact numbers shown above are illustrative of the expected format only — actual values will depend on the temperature set in your own simulation.

8. Simulation Procedure (Tinkercad Circuits)

1. Go to Tinkercad (tinkercad.com), sign in, and open the Circuits section.
2. Click Create new Circuit to open a blank design canvas.
3. From the components panel, drag an Arduino Uno onto the canvas.
4. Drag a Temperature Sensor [TMP36] onto the canvas, near the Arduino.
5. Drag an LED and a Resistor (set its value to 220 Ω) onto the canvas.
6. Wire the components exactly as shown in the circuit diagram: TMP36 VCC → 5V, TMP36 GND → GND, TMP36 OUT → A0, Digital Pin 8 → Resistor → LED Anode, LED Cathode → GND.
7. Click on the Arduino Uno and open the Code editor. Switch the mode from “Blocks” to “Text”.
8. Delete any placeholder code and paste in the complete contents of src/temperature_sensor.ino.
9. Click the Start Simulation button. Open the Serial Monitor from the code panel to view live temperature readings.
10. Click on the TMP36 sensor in the simulation and drag its temperature slider above and below 30°C to confirm the LED turns ON and OFF correctly.

9. Conclusion

This project demonstrates a simple and reliable IoT-based temperature alert system using an Arduino Uno, a TMP36 temperature sensor, and an LED. It shows the fundamental IoT workflow of sensing real-world data, processing it digitally, and triggering an automatic action — all without requiring any physical hardware, since the entire circuit can be built and tested in Tinkercad Circuits. This

sense-decide-act pattern is the foundation for more advanced IoT applications such as smart home automation, industrial monitoring, and healthcare alert systems.

CodeAlpha IoT Internship — Task 2: Sensor-Based Simulation — Temperature-Based IoT Alert System